AIM825 Course Project Report

Multimodal Visual Question Answering with Amazon Berkeley Objects Dataset

Harsh Kumar Chandrakar (MT2024054)
Akshay Sharma (MT2024016)
Shashank Vyas (MT2024141)

International Institute of Information Technology, Bangalore

May 18, 2025

Abstract

This report details the development and evaluation of a Visual Question Answering (VQA) system using the Amazon Berkeley Objects (ABO) dataset. A key contribution of this project is the creation of a new VQA dataset tailored to the ABO images, generated using a novel dual-prompting strategy with both a locally run LLaVA model and the API-based Gemini-2.0-Flash model to balance question descriptiveness and accuracy. The dataset focuses on single-word answers. We evaluated a baseline BLIP-2 model and subsequently fine-tuned it using Low-Rank Adaptation (LoRA) with specific hyperparameters to improve performance on our curated VQA task. The performance of both models was assessed using Exact Match, Partial Match, BLEU, and BERTScore metrics. Results indicate that LoRA fine-tuning improved Exact Match accuracy from 31.45% to 34.45% and showed a slight improvement in Partial Match from 45.98% to 47.42%. However, it led to a decrease in BLEU and BERTScore F1 metrics. This report discusses the methodologies for data curation, model fine-tuning, quantitative and qualitative results, and the structure of the inference script developed.

Contents

1 Introduction

Visual Question Answering (VQA) is a challenging multimodal task that requires an AI system to understand an image and a natural language question about the image, and generate an appropriate natural language answer. VQA has numerous applications, including aiding visually impaired users, robotics, and interactive image retrieval. The complexity of VQA lies in the need to bridge the gap between visual perception and natural language understanding, often requiring commonsense reasoning and detailed image interpretation.

The Amazon Berkeley Objects (ABO) dataset, with its vast collection of product listings and catalog images, provides a rich source for developing VQA systems in the e-commerce domain. However, pre-existing VQA annotations for this dataset are not readily available for specialized tasks. This project addresses this by first curating a new VQA dataset specifically from the ABO (small variant) images.

The primary objectives of this project were:

1. To curate a new VQA dataset from the ABO images, focusing on single-word answers, using multimodal models for question-answer generation.
2. To evaluate the performance of a pre-trained baseline multimodal model (BLIP-2) on this curated dataset.
3. To fine-tune the baseline model using Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning technique, to adapt it better to our specific VQA task.
4. To compare the performance of the fine-tuned model against the baseline using standard VQA metrics: Exact Match, Partial Match, BLEU, and BERTScore.
5. To develop an inference script capable of loading the fine-tuned model and performing VQA on new image-question pairs.

This report is structured as follows: Section 2 details the data curation process. Section 3 describes the baseline model, the fine-tuning approach using LoRA, and the experimental setup. Section 4 presents the evaluation metrics for both models, including a comparative analysis and qualitative examples. Section 5 describes the inference script. Finally, Section 6 concludes the report and suggests potential avenues for future work. The project adhered to constraints including using free cloud GPUs (Kaggle) and ensuring the final model size was under 7 billion parameters.

2 Data Curation

A significant part of this project was the creation of a Visual Question Answering dataset tailored to the Amazon Berkeley Objects (ABO) dataset (small variant). The process, primarily detailed in the `data_curation.ipynb` notebook, focused on generating image-question-answer triplets where answers were single words.

2.1 Source Dataset

The ABO dataset (small variant) was used as the source for images. This dataset contains numerous product listings with associated images, providing a diverse set of objects for VQA.

2.2 Question-Answer Generation

Multimodal Large Language Models (MLLMs) were employed to generate questions and answers for the ABO images. A unique dual-prompting strategy was adopted, utilizing both the Gemini-2.0-Flash API and a locally run LLaVA model. This approach aimed to balance the strengths of both models:

- **Gemini-2.0-Flash API:** Used for its accuracy, it was prompted with shorter, more direct queries to generate precise questions and answers.
- **Local LLaVA Model:** This model allowed for more detailed and descriptive prompts. This was particularly useful for generating questions that could differentiate between visually similar items (e.g., different types of phone cases or specific product features). The prompts for LLaVA were crafted to encourage more descriptive questions.

The specific prompts used for each model are detailed within the `data_curation.ipynb` notebook. This hybrid approach was a novel aspect of our data curation, attempting to maximize both the quality and diversity of the generated VQA pairs. The generated answers were predominantly single words, simplifying the VQA task to a more constrained output format.

2.3 Final Curated Dataset

The output of the data curation process is the `simplified_vqa_dataset.csv` file. This CSV file contains the following metadata for each entry:

- `image_path`: The path or identifier for the image from the ABO dataset.
- `generated_question`: The generated natural language question about the image.
- `generated_answer`: The generated single-word answer to the question.

This curated data set focuses on single-word answers and does not include pre-defined multiple-choice options. The data set was then split into three sets using the `VR_Assignment2_HarshChandrakar_MT2024054/data_split.py` script to generate the VQA dataset parallelly among three of us. The presence of files such as `balanced*.csv` in the project directory indicates that data balancing techniques were explored or applied during the dataset preparation phase, the specifics of which are detailed in the `data_curation.ipynb` or related scripts.

3 Methodology

This section outlines the chosen baseline model, the fine-tuning approach using LoRA, and the experimental setup.

3.1 Baseline Model: BLIP-2

The BLIP-2 model was selected as the baseline for our VQA task. BLIP-2 (Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models) is a powerful vision-language model known for its efficiency and strong performance on various VQA and image captioning benchmarks. It achieves this by aligning a frozen pre-trained image encoder with a frozen large language model using a lightweight Querying Transformer (Q-Former). The baseline evaluation was performed using the code provided in `baseline_blip2.ipynb` (referring to the latest version provided). This involved loading the pre-trained BLIP-2 model (`Salesforce/blip2-flan-t5-xl`) and evaluating its zero-shot performance on our curated test set from `simplified_vqa_dataset.csv`.

3.2 Fine-Tuning Approach: LoRA

To adapt the BLIP-2 model (`Salesforce/blip2-flan-t5-xl`) more specifically to our curated ABO VQA dataset, we employed Low-Rank Adaptation (LoRA). LoRA is a parameter-efficient fine-tuning (PEFT) technique that freezes the pre-trained model weights and injects trainable rank decomposition matrices into selected layers of the Transformer architecture. This significantly reduces the number of trainable parameters compared to full fine-tuning, making it feasible to fine-tune large models on limited compute resources (Kaggle, as per project constraints).

The fine-tuning process using LoRA is documented in the `VR_Assignment2_HarshChandrakar_MT2024054/lora-blip2_finetuned.ipynb` notebook. The specific LoRA configuration and training hyperparameters used are detailed below:

LoRA Configuration (PeftConfig): The LoRA adapter was configured with the following parameters:

- **Rank (r):** 4
- **Alpha ($_\alpha$):** 16
- **Target Modules:** `["q", "v"]` (targeting query and value projection layers)
- **Dropout ($_{dropout}$):** 0.05
- **Bias:** "none"
- **Task Type:** `TaskType.SEQ_2_SEQ_LM`
- **Modules to Save:** `["lm_head"]`

Training Hyperparameters and Setup: The model was fine-tuned using a custom training loop with the following settings:

- **Base Model Loading:** The base `Salesforce/blip2-flan-t5-xl` model was loaded with `torch_dtype=torch.float16`, `device_map="balanced"`, `offload_folder="/kaggle/working/offload"` `offload_state_dict=True`, and `low_cpu_mem_usage=True` to manage memory effectively.
- **Optimizer:** AdamW
- **Learning Rate:** 5e-6

- **Weight Decay:** 0.01
- **AdamW Epsilon (ϵ):** 1e-8
- **Number of Training Epochs:** 2
- **Batch Size (per device, DataLoader):** 4
- **Gradient Accumulation Steps:** 2 (Effective batch size: $4 \times 2 = 8$)
- **Scheduler:** Linear schedule with warmup
- **Warmup Steps:** 10
- **Mixed-Precision Training (fp16):** Enabled (using 'torch.cuda.amp.GradScaler' and 'torch.autocast')
- **Training Data:** A subset of 18,000 samples from the training split of `simplified_vqa_dataset.csv`.
- **Checkpoint Directory (per epoch):** `"/kaggle/working/blip2-lora-epoch-<epoch_num>"`
- **Final Adapter Output Directory:** `"/kaggle/working/blip2-lora-final"`

The model was fine-tuned on the training split of our `simplified_vqa_dataset.csv`.

3.3 Experimental Setup

- **Hardware:** All training and fine-tuning were conducted on free cloud GPU resources (Kaggle), utilizing features like balanced device mapping for multi-GPU usage if available, adhering to project constraints.
- **Software:** The primary libraries used include PyTorch, Hugging Face Transformers (for BLIP-2 model and LoRA implementation via PEFT library), Pandas for data manipulation, and various metric calculation libraries (e.g., NLTK for BLEU, 'bert-score' for BERTScore). The `VR_Assignment2_HarshChandrakar_MT2024054/requirements.txt` file lists the specific Python packages and their versions.
- **Dataset Split:** The `simplified_vqa_dataset.csv` was split into training, validation, and test sets to ensure robust evaluation.

4 Results and Discussion

This section presents the quantitative results for both the baseline BLIP-2 model and the LoRA fine-tuned BLIP-2 model. A comparative analysis and qualitative discussion follow.

4.1 Evaluation Metrics

The models were evaluated using the following standard VQA metrics:

- **Exact Match (EM):** The percentage of predicted answers that exactly match the ground truth answer.
- **Partial Match (PM):** The percentage of predicted answers that contain the ground truth answer as a substring, or vice-versa, or have significant overlap.
- **BLEU Score (B):** A precision-based metric that measures n-gram overlap between predicted and ground truth answers.
- **BERTScore (F1):** Measures semantic similarity between predicted and ground truth answers using contextual embeddings from BERT. The F1-score is reported.

The results from the updated `metrics_final.txt` (baseline) and `FinetunedMetrics.txt` (fine-tuned) are summarized in Table 1.

4.2 Quantitative Results

Table 1: Comparison of VQA Metrics: Baseline BLIP-2 vs. LoRA Fine-tuned BLIP-2

Model	Exact Match (%)	Partial Match (%)	BLEU Score	BERTScore F1
Baseline BLIP-2	31.45	45.98	0.4358	0.8862
Fine-tuned BLIP-2 (LoRA)	34.45	47.42	0.0880	0.6766

4.3 Discussion of Results

The quantitative results presented in Table 1 reveal several interesting points:

- **Exact Match and Partial Match Improvement:** The LoRA fine-tuned BLIP-2 model shows an improvement in Exact Match accuracy, increasing from 31.45% (baseline) to 34.45%. Similarly, Partial Match also saw a slight improvement from 45.98% (baseline) to 47.42% for the fine-tuned model. This suggests that fine-tuning successfully adapted the model to better predict answers that are lexically identical or closely related to the ground truth single-word answers in our curated dataset.
- **Decrease in BLEU and BERTScore:** Conversely, the fine-tuned model performed notably worse than the baseline on BLEU Score and BERTScore F1.
  - BLEU Score saw a significant drop from 0.4358 (baseline) to 0.0880 (fine-tuned).
  - BERTScore F1 also decreased substantially from 0.8862 (baseline) to 0.6766 (fine-tuned).
- **Interpreting Metric Discrepancies:** This divergence in metric trends (EM and PM up, BLEU and BERTScore F1 down) is a key finding. Several factors could explain this observation:

1. **Specificity vs. Generality:** Fine-tuning on a dataset with predominantly single-word answers may have pushed the model towards generating very concise outputs. While this benefits Exact Match and, to a lesser extent, Partial Match, it could penalize answers that might have broader semantic similarity or n-gram overlap if they are not very close lexical matches. The baseline model, being more general, might have produced answers that, while less often exact, scored higher on these n-gram or embedding-based semantic metrics.
2. **Overfitting to Answer Style:** The fine-tuned model may have overfitted to the specific style or vocabulary of the single-word answers in the training set. This could lead to higher exact/partial matches when that style is appropriate but lower scores when a slight variation would be considered correct by BLEU or BERTScore.
3. **Nature of Single-Word Answers and Metric Sensitivity:** For single-word answers, BLEU scores can be very sensitive; if the single word is not an exact match, BLEU can drop dramatically. BERTScore, while semantic, could also be affected if the fine-tuned model's vocabulary or phrasing for non-exact (but perhaps still relevant) answers becomes more restricted or less diverse than the baseline. The slight improvement in Partial Match suggests the fine-tuned model is still capturing some lexical closeness even when not exact.
4. **Dataset Characteristics:** The fine-tuning dataset's characteristics may have influenced the model to prioritize lexical precision, benefiting EM and PM, but potentially at the cost of performance on metrics that reward broader n-gram or semantic overlap if the generated answers deviate from the ground truth even slightly.

4.4 Qualitative Analysis

To further understand the model behaviors, a qualitative analysis of predictions is necessary. For the **baseline model**, files like `blip2_baseline_results_final.csv` (from the latest run), `exact_matches_final.csv`, and `partial_matches_final.csv` provide examples of its outputs. Predictions from the **fine-tuned model** were saved to `VR_Assignment2_HarshChandrakar_MT2024054/results.csv` for analysis (as indicated by the notebook structure, although the evaluation script in the fine-tuning notebook might save to 'enhanced_results.csv').

Examining specific examples would involve looking at:

- Cases where the fine-tuned model achieved an exact or partial match but the baseline did not, and vice-versa.
- Cases where the baseline had a high BERTScore or BLEU, but the fine-tuned model did not, despite potentially achieving an exact or partial match.
- Common error types for both models.

The unique data curation strategy, combining precise Gemini outputs with descriptive LLaVA outputs, created a diverse set of questions. The model's ability to handle this diversity, especially after fine-tuning for single-word answers, would be an interesting subject for qualitative observation.

5 Inference Script

An inference script, `VR_Assignment2_HarshChandrakar_MT2024054/inference.py`, was developed to allow for VQA on new data using the fine-tuned model. The script is designed to perform the following functions:

1. **Load Model and Tokenizer:**
   - Load the pre-trained BLIP-2 model architecture (`Salesforce/blip2-flan-t5-xl`).
   - Load the fine-tuned LoRA weights (adapter) into the base model using the PEFT library. The notebook indicates the final adapter was saved in a directory like `"/kaggle/working/blip2-lora-final"` or potentially pushed to Hugging Face Hub under an account like `"Magneto76/lora_blip2"` (as seen in later cells of the fine-tuning notebook for loading/inference testing).
   - Load the associated processor from the same location as the fine-tuned adapter or base model.
2. **Input Processing:**
   - Accept an image (e.g., path to an image file) and a natural language question as input.
   - Preprocess the input image to the format expected by the BLIP-2 image encoder.
   - Tokenize the input question using the model's processor.
3. **Inference/Answer Generation:**
   - Feed the processed image and question to the fine-tuned BLIP-2 model.
   - Generate an answer using model generation parameters optimized for concise outputs (e.g., 'max_new_tokens', 'num_beams', 'early_stopping', 'repetition_penalty', 'length_penalty' as seen in the inference testing part of the fine-tuning notebook).
   - Decode the generated tokens back into a human-readable text answer.
4. **Output:**
   - Return the generated and processed answer. The inference part of the fine-tuning notebook includes significant post-processing logic to refine the raw answer (lowercase, remove prefixes/suffixes, normalize terms, limit word count).

The script includes argument parsing to accept image paths and questions from the command line. The specific implementation details are found in `VR_Assignment2_HarshChandrakar_MT2024054/inference.py`.

6 Conclusion and Future Work

This project successfully addressed the task of developing a Visual Question Answering system for the Amazon Berkeley Objects dataset. Key achievements include:

- The curation of a new VQA dataset (`simplified_vqa_dataset.csv`) tailored to ABO images with single-word answers, using a novel dual-prompting strategy with Gemini and LLaVA to balance accuracy and question descriptiveness.
- Evaluation of a baseline BLIP-2 model on this dataset.
- Successful fine-tuning of the BLIP-2 model using LoRA with the specified configurations and training hyperparameters, demonstrating parameter-efficient adaptation.
- A comparative analysis of the baseline and fine-tuned models. Fine-tuning led to improvements in Exact Match (31.45% to 34.45%) and Partial Match (45.98% to 47.42%). However, a notable decrease was observed in BLEU Score and BERTScore F1 for the fine-tuned model compared to the baseline.
- Development of an inference script for the fine-tuned model.

The mixed impact of fine-tuning on different metrics is a significant finding. While lexical precision for single-word answers improved (EM, PM), the performance on metrics sensitive to n-gram overlap (BLEU) and broader semantic similarity (BERTScore F1) declined. This highlights the nuanced trade-offs in model adaptation and the importance of considering a diverse set of evaluation metrics in the context of specific task constraints, such as the single-word answer format.

Future Work: Several avenues could be explored to build upon this work:

- **Expanded Dataset with Options:** Extend the curated dataset to include multiple-choice options, making the task more complex and potentially allowing for different evaluation strategies.
- **Refined LoRA Configurations:** Experiment with different LoRA configurations (e.g., varying ranks, alpha values, or targeting different/more modules) to potentially improve performance across all metrics.
- **Qualitative Error Analysis:** Conduct a more in-depth qualitative error analysis to better understand the reasons for the decrease in BLEU and BERTScore F1 after fine-tuning and identify common failure modes.
- **Alternative Base Models:** Explore other state-of-the-art vision-language models as baselines or for fine-tuning.
- **Multi-word Answers:** Adapt the system to handle multi-word or sentence-based answers, which would make metrics like BLEU and BERTScore more directly indicative of overall answer quality.
- **Exploring Different Prompting Strategies for Curation:** Further refine the prompting strategies for data curation to enhance the quality and diversity of the generated VQA pairs.

Overall, this project provides valuable insights into the process of dataset curation for VQA, parameter-efficient fine-tuning, and the complexities of model evaluation in specialized VQA tasks.

References

[1] Li, J., Li, D., Savarese, S., & Hoi, S. (2023). *BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models*. arXiv preprint arXiv:2301.12597.

[2] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685.

[3] Amazon Berkeley Objects (ABO) Dataset. (Accessed 2024-2025). Available at relevant dataset source (e.g., https://amazon-berkeley-objects.s3.amazonaws.com/index.html)

[4] Google. Gemini API Documentation. (Accessed 2024-2025). Google AI for Developers.

[5] Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). *Visual Instruction Tuning (LLaVA)*. arXiv preprint arXiv:2304.08485.

[6] Hugging Face PEFT Library. (Accessed 2024-2025). https://huggingface.co/docs/peft